3. Object Detection

Enter the Docker container, and then execute the tutorial commands within Docker to launch the program.

The primary objective of this section is to demonstrate how to utilize the `dnn` module within OpenCV to load and deploy a pre-trained object detection network. However, this process does impose specific requirements regarding the OpenCV version used.

Currently, there are three main approaches to performing object detection using deep learning:

- Faster R-CNNs
- You Only Look Once (YOLO)
- Single Shot Detectors (SSDs)

Faster R-CNNs are arguably the most widely recognized deep learning-based neural networks in the field. However, this approach is technically complex to grasp (particularly for newcomers to deep learning), challenging to implement, and difficult to train.

Furthermore, even with the "Faster" implementation of R-CNNs (where "R" stands for Region Proposal), the algorithm remains relatively slow, typically operating at approximately 7 FPS.

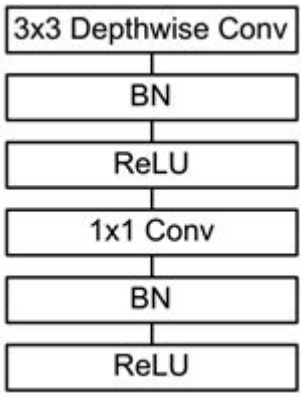
If speed is our priority, we can turn to YOLO. It is exceptionally fast, capable of achieving speeds of 40–90 FPS on a Titan X GPU, with its fastest variant potentially reaching up to 155 FPS. However, the drawback of YOLO is that its detection accuracy still leaves room for improvement.

SSDs, originally developed by Google, can be viewed as striking a balance between the two aforementioned approaches. Compared to Faster R-CNNs, the SSD algorithm is more straightforward; compared to YOLO, it offers greater accuracy.

3.1. Model Architecture

The core innovation of MobileNet lies in replacing traditional *standard convolutions* with *depthwise separable convolutions* to address issues regarding computational efficiency and parameter count within convolutional networks. The MobileNet architecture is built upon these depthwise separable convolutions, which allow a standard convolution to be decomposed into two distinct operations: a *depthwise convolution* and a *pointwise convolution* (using a 1×1 kernel). **The depthwise convolution applies a separate filter to each individual input channel, while the 1×1 pointwise convolution is used to combine the outputs generated by the channel-wise convolutions.**

Batch Normalization (BN) is incorporated as a fundamental component within MobileNet. During each iteration of Stochastic Gradient Descent (SGD), BN performs a normalization step to ensure that the resulting output signals—across all dimensions—have a mean of 0 and a variance of 1. Generally, when training neural networks, if one encounters issues such as extremely slow convergence or training instability (e.g., due to exploding gradients), applying Batch Normalization can serve as an effective remedy. Furthermore, even under standard operating conditions, incorporating BN can help accelerate the training process and enhance the overall accuracy of the model. Furthermore, the model employs the ReLU activation function; thus, the basic structure of a depthwise separable convolution is illustrated in the figure below:



The MobileNets network is constructed by combining numerous depthwise separable convolutions of the type shown in the figure above. Its specific network architecture is depicted in the figure below:

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s1	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

3.2. Code Analysis

List of Recognizable Objects

```
[person, bicycle, car, motorcycle, airplane, bus, train,
 truck, boat, traffic light, fire hydrant, street sign,
 stop sign, parking meter, bench, bird, cat, dog, horse,
 sheep, cow, elephant, bear, zebra, giraffe, hat, backpack,
 umbrella, shoe, eye glasses, handbag, tie, suitcase,
 frisbee, skis, snowboard, sports ball, kite, baseball bat,
```

```
 baseball glove, skateboard, surfboard, tennis racket,
 bottle, plate, wine glass, cup, fork, knife, spoon, bowl,
 banana, apple, sandwich, orange, broccoli, carrot, hot dog,
 pizza, donut, cake, chair, couch, potted plant, bed, mirror,
 dining table, window, desk, toilet, door, tv, laptop, mouse,
 remote, keyboard, cell phone, microwave, oven, toaster,
 sink, refrigerator, blender, book, clock, vase, scissors,
 teddy bear, hair drier, toothbrush]
```

Load the category [object_detection_coco.txt], import the model [frozen_inference_graph.pb], and specify the deep learning framework [TensorFlow].

Provided source code package path:

```
dofbot_pro_vision/dofbot_pro_vision/detect_object.py
```

```
# Load COCO class names
with
open('/root/dofbot_ws/src/dofbot_pro_vision/config/object_detection_coco.txt',
'r') as f:
            self.class_names = f.read().split('\n')
# Display different colors for different targets
self.COLORS = np.random.uniform(0, 255, size=(len(self.class_names), 3))


# Load DNN Image Model (Jetson Nano Board)
self.model =
cv.dnn.readNet(model='/root/dofbot_ws/src/dofbot_pro_vision/config/frozen_infere
nce_graph.pb',
config='/root/dofbot_ws/src/dofbot_pro_vision/config/ssd_mobilenet_v2_coco.txt',
framework='TensorFlow')
```

The image was imported, its height and width were extracted, a 300x300 pixel blob was computed, and this blob was passed into the neural network.

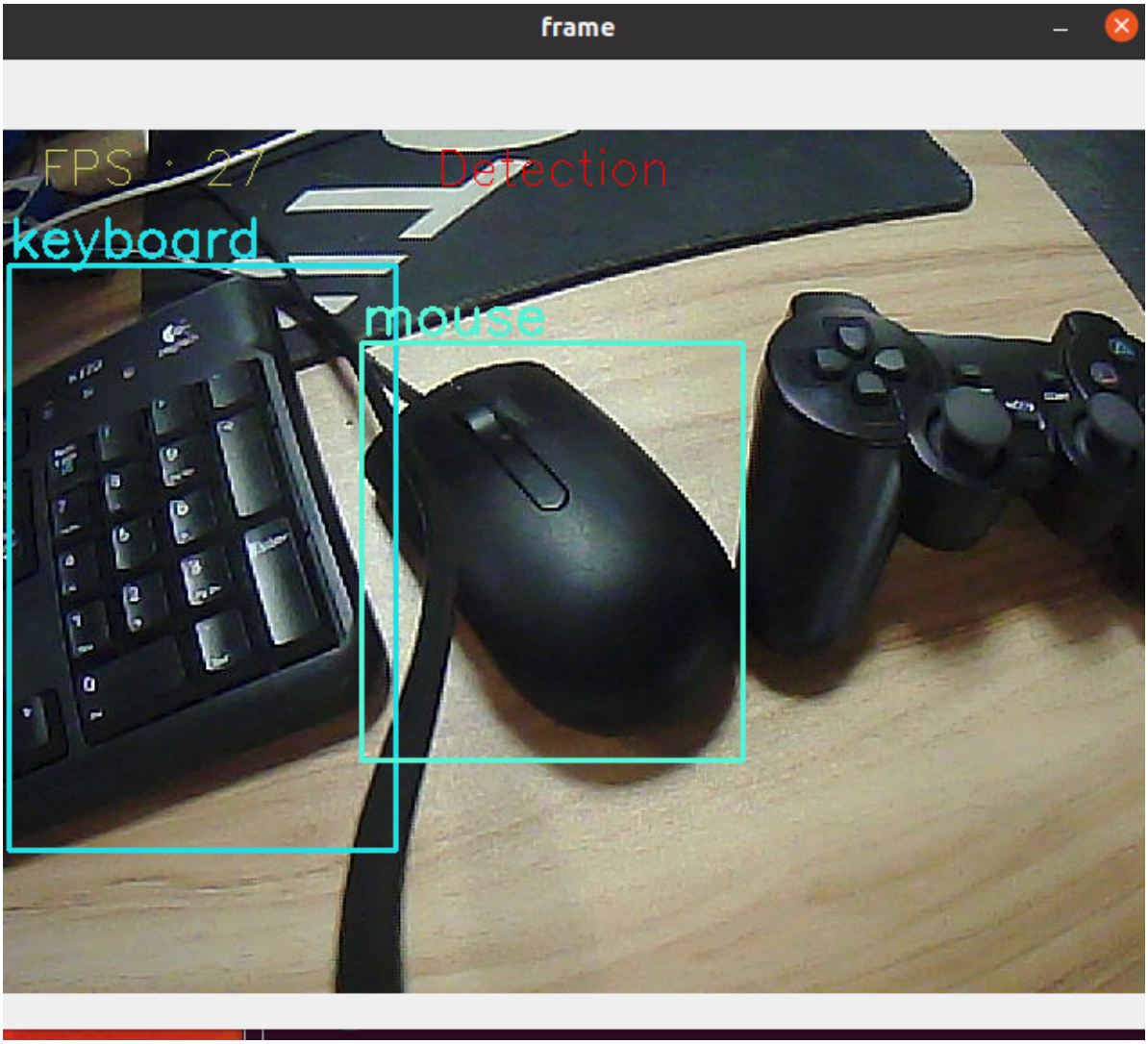
```
def Target_Detection(image):
    image_height, image_width, _ = image.shape
    # Create a Blob from an Image
    blob = cv.dnn.blobFromImage(image=image, size=(300, 300), mean=(104, 117,
123), swapRB=True)
    model.setInput(blob)
    output = model.forward()
    # Iterate through each detection
    for detection in output[0, 0, :, :]:
        # Extract detection confidence
        confidence = detection[2]
        # Draw bounding boxes only when the detection confidence exceeds a
certain threshold; otherwise, skip.
        if confidence > .4:
            # Get the class ID
            class_id = detection[1]
            # Map class IDs to classes
            class_name = class_names[int(class_id) - 1]
            color = COLORS[int(class_id)]
```

```
            # Get bounding box coordinates
            box_x = detection[3] * image_width
            box_y = detection[4] * image_height
            # Get the width and height of the bounding box
            box_width = detection[5] * image_width
            box_height = detection[6] * image_height
            # Draw a rectangle around each detected object.
            cv.rectangle(image, (int(box_x), int(box_y)), (int(box_width),
int(box_height)), color, thickness=2)
            # Write the class name text on the detected objects.
            cv.putText(image, class_name, (int(box_x), int(box_y - 5)),
cv.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    return image
```

3.3. Launching

```
# Launch the camera (enter this command within the Docker container terminal):
ros2 launch dofbot_info camera_arm_kin.launch.py
# Launch the object detection program (enter this command within the Docker
container terminal):
ros2 run dofbot_pro_vision detect_object
```

Camera Feed:


frame
FPS : 27
Detection
keyboard
mouse